

Boosting: Gradient Boosting, XGBoost e LightGBM

Corrigir o erro do modelo anterior, uma árvore rasa de cada vez

Random Forest treina árvores em paralelo e tira a média — cada árvore não sabe da existência das outras. Boosting inverte a lógica: treina árvores em sequência, cada uma focada em corrigir exatamente o erro que o ensemble acumulado até ali ainda comete. Esse ajuste sequencial, formalizado como gradiente descendente no espaço de funções, é o que sustenta XGBoost e LightGBM — o estado da arte em dados tabulares. Este material constrói Gradient Boosting do zero, cobre os avanços de engenharia das bibliotecas modernas, e formaliza early stopping.

Nível: Avançado — encerra o tema de Aprendizado Supervisionado

Pre-requisitos: Árvores de Decisão; Bagging e Random Forest; Cálculo e Gradiente Descendente (tema 2)

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-gradient-boosting-do-zero · 02-xgboost-e-lightgbm · 03-tuning-e-early-stopping · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.1

Sumário

1. Por que este módulo existe	3
1.1 O que você vai conseguir fazer ao final	3
2. Gradient Boosting: gradiente descendente no espaço de funções	5
2.1 Um exemplo numérico: as duas primeiras rodadas, na mão	5
3. Taxa de aprendizado e o número de árvores	7
4. XGBoost e LightGBM: os avanços de engenharia	9
5. Early stopping: decidir M sem vazar o teste	10
6. Erros que custam caro — checklist	11
7. Para ir além	12

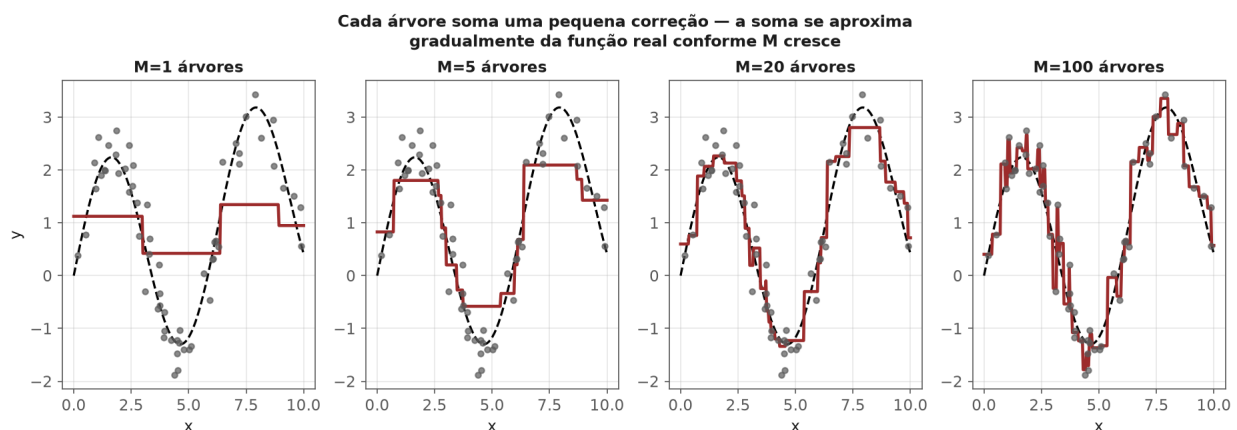
1. Por que este módulo existe

Random Forest (módulo anterior) reduz variância combinando árvores **independentes** — cada uma treinada sem saber o que as outras fazem, numa amostra bootstrap diferente. Isso funciona bem quando o problema é reduzir a variância de um modelo já razoável. Boosting resolve um problema diferente: reduzir o **viés**, construindo o ensemble **sequencialmente**, onde cada nova árvore é treinada especificamente para corrigir o que o ensemble acumulado até aquele ponto ainda erra. É essa mudança de filosofia — paralelo e independente vs. sequencial e corretivo — que torna boosting, bem ajustado, o método de maior desempenho em dados tabulares na maioria das competições e aplicações de mercado.

ANALOGIA

Random Forest é como pedir a mil especialistas independentes para avaliar um caso e tirar a média dos palpites. Boosting é como um processo de revisão em cadeia: o primeiro especialista dá um diagnóstico inicial, o segundo olha **especificamente para os casos que o primeiro errou** e tenta corrigi-los, o terceiro foca no que os dois primeiros ainda erram juntos, e assim por diante. Cada novo participante tem uma tarefa cada vez mais específica: consertar o que sobrou.

A figura a seguir mostra essa ideia num problema de regressão simples: com apenas 1 árvore, o "ensemble" é quase uma reta grosseira; com 5, já capta a forma geral da curva; com 20 e 100, o ajuste refina cada vez mais os detalhes que as árvores anteriores ainda erravam.



A soma de árvores rasas se aproxima gradualmente da função real: cada nova árvore corrige o que a soma anterior ainda erra.

1.1 O que você vai conseguir fazer ao final

- Implementar Gradient Boosting do zero, entendendo por que é gradiente descendente no espaço de funções, não de parâmetros.
- Explicar o papel da taxa de aprendizado (*shrinkage*) e por que boosting

exige controle cuidadoso do número de rounds.

- Descrever os avanços de XGBoost e LightGBM sobre o Gradient Boosting clássico, e quando cada biblioteca é preferível.
 - Implementar early stopping corretamente, sem vaziar o conjunto de teste.
-

2. Gradient Boosting: gradiente descendente no espaço de funções

Em vez de otimizar parâmetros β (temas anteriores), Gradient Boosting otimiza diretamente a **função de previsão** $F(x)$, construindo-a como uma soma de árvores:

$$F_M(x) = \sum_{m=1}^M \eta \cdot h_m(x)$$

onde cada h_m é uma árvore rasa (um "aprendiz fraco") e η é a taxa de aprendizado. O algoritmo:

- 1 Comece com uma previsão constante simples, $F_0(x)$ (a média do alvo, por exemplo).
- 2 Para $m = 1, \dots, M$: calcule o **gradiente negativo da perda** em relação à previsão atual, para cada exemplo — isso é literalmente um **resíduo** no caso de erro quadrático (tema 2: o gradiente de $\frac{1}{2}(y - F)^2$ em relação a F é $-(y - F)$).
- 3 Treine uma nova árvore h_m para prever esse gradiente negativo (o "erro residual" a corrigir).
- 4 Atualize: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$.

FORMULARIO

Essa é a mesma lógica do gradiente descendente do tema 2, módulo 3 — só que em vez de dar um passo no espaço de parâmetros ($\beta \leftarrow \beta - \eta \nabla \mathcal{L}$), o passo é dado no **espaço de funções** ($F \leftarrow F + \eta \cdot h$, onde h aproxima a direção de máxima redução da perda). É por isso que o nome "gradient boosting" não é uma metáfora — é gradiente descendente, aplicado a um objeto matemático diferente (uma função, não um vetor de números).

2.1 Um exemplo numérico: as duas primeiras rodadas, na mão

Considere 4 exemplos com alvo $y = [10, 12, 20, 22]$. F_0 começa como a média: $F_0(x) = 16$ para todos. Os resíduos (gradiente negativo, para erro quadrático) da rodada 1 são $y - F_0 = [-6, -4, 4, 6]$.

Uma árvore rasa h_1 é treinada para prever **esses resíduos** — por exemplo, aprendendo a distinguir os dois primeiros exemplos (resíduo médio -5) dos dois últimos (resíduo médio $+5$). Com $\eta = 0,3$:

$$F_1(x) = F_0(x) + 0,3 \times h_1(x)$$

Para os dois primeiros exemplos, $F_1 = 16 + 0,3 \times (-5) = 14,5$; para os dois últimos, $F_1 = 16 + 0,3 \times 5 = 17,5$. O erro caiu (de $|10 - 16| = 6$ para $|10 - 14,5| = 4,5$), mas não foi eliminado de propósito — $\eta = 0,3$ garante que **cada rodada dá só um passo parcial**, deixando erro suficiente para as próximas árvores continuarem corrigindo. Repetir esse processo dezenas ou centenas de vezes é exatamente o que a figura anterior ilustra visualmente.

NOTA

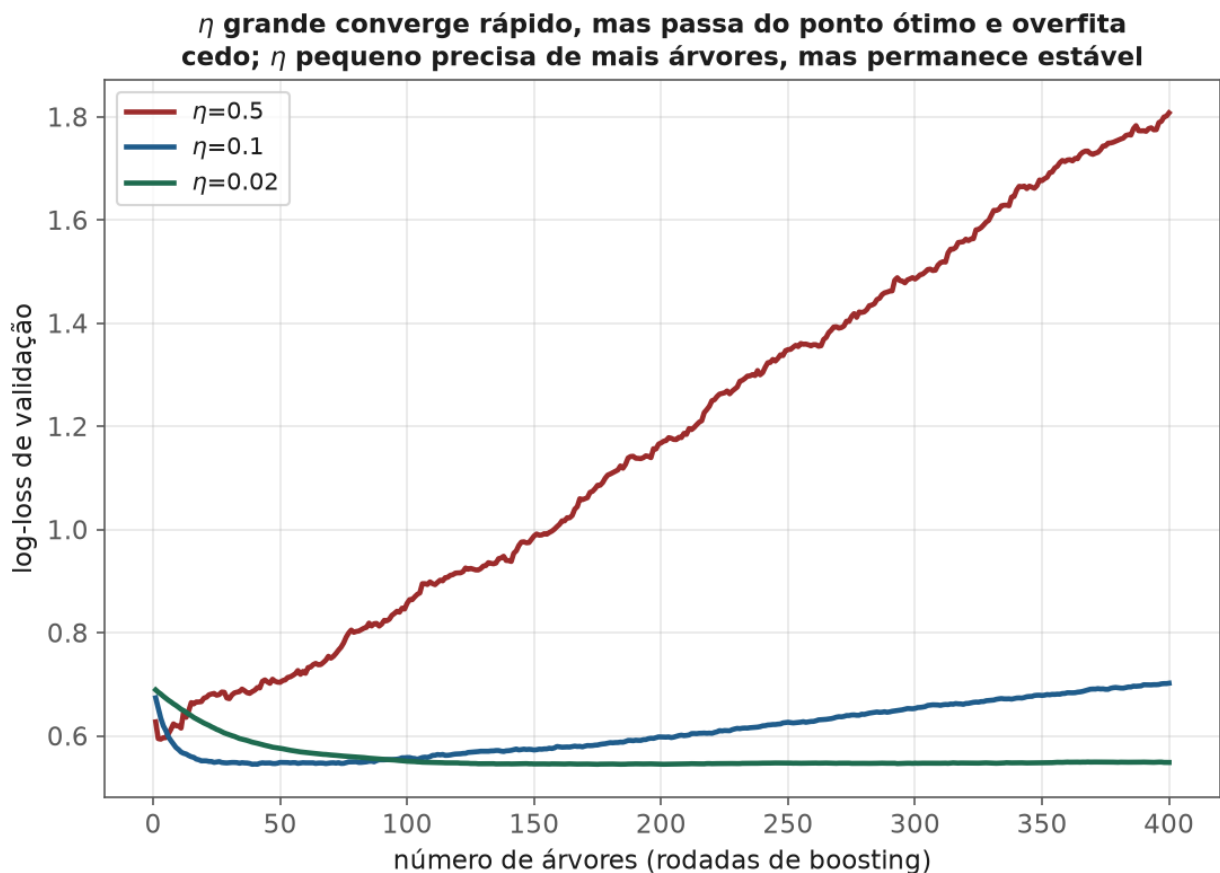
Para erro quadrático, o gradiente negativo é o resíduo $(y - F_{m-1}(x))$ — daí a intuição popular de "cada árvore aprende o erro da anterior". Para outras funções de perda (entropia cruzada, por exemplo), o "resíduo" generalizado ainda é o gradiente negativo da perda, mas não coincide exatamente com $y - \hat{y}$.

3. Taxa de aprendizado e o número de árvores

ARMADILHA COMUM

η pequeno exige mais árvores (M maior) para atingir o mesmo ajuste, mas produz um ensemble mais robusto e menos propenso a overfitting — o mesmo trade-off do tema 2, módulo 3, reaparecendo aqui entre "quantidade de passos" e "tamanho de cada passo". A combinação η pequeno + M grande, com early stopping (adiante) para decidir o M real, é a prática padrão de mercado.

A figura abaixo mostra esse trade-off diretamente: com $\eta = 0,5$ (vermelho), a perda de validação já começa a piorar antes da rodada 20 — o passo é grande demais e o modelo overfita rápido. Com $\eta = 0,1$ (azul), o modelo tem uma janela maior de rodadas úteis antes de overfitar. Com $\eta = 0,02$ (verde), o modelo permanece estável por todas as 400 rodadas simuladas, sem sinal de overfitting.

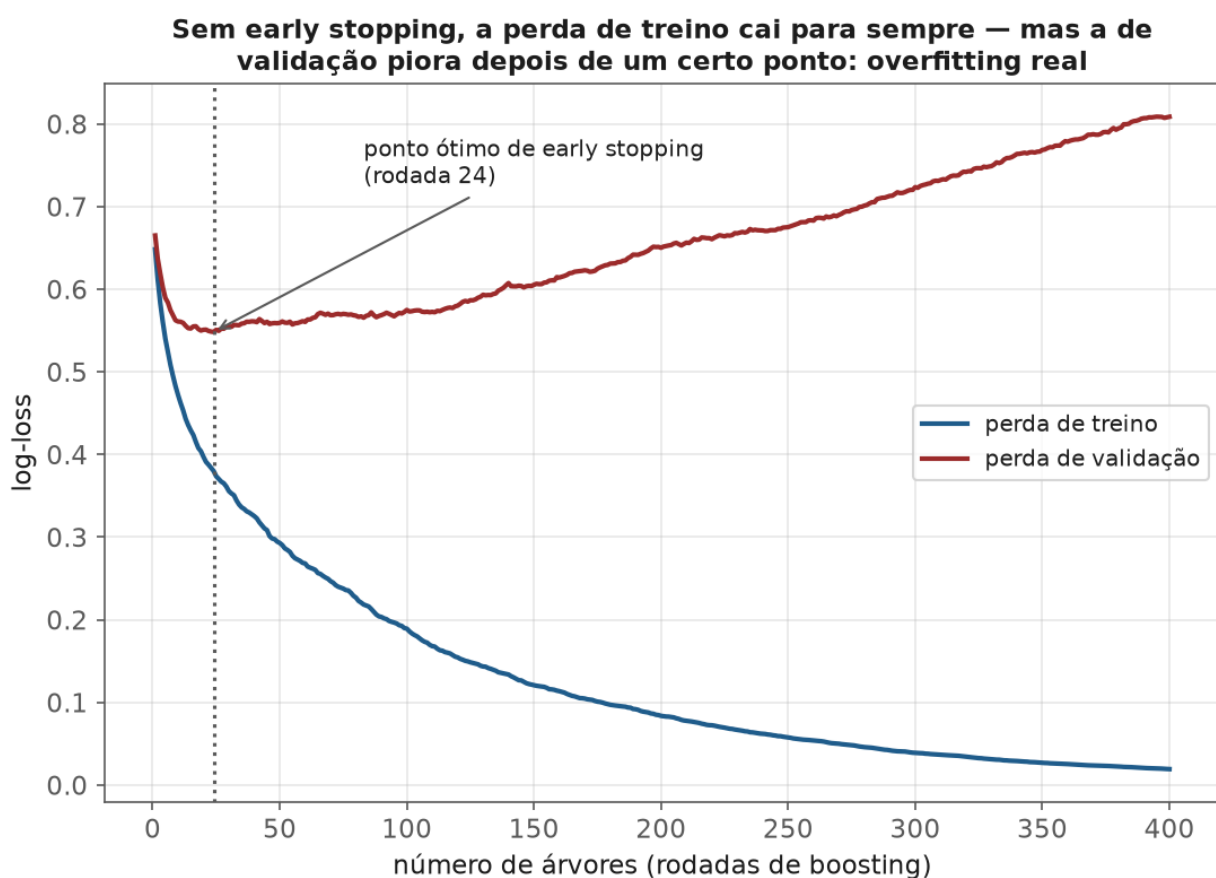


Taxa de aprendizado grande converge rápido mas overfita cedo; taxa pequena precisa de mais árvores, mas permanece estável por muito mais rodadas.

ARMADILHA COMUM

Ao contrário de Random Forest, onde adicionar mais árvores **quase nunca piora** o resultado, em boosting **adicionar árvores demais causa overfitting real** — cada nova árvore continua tentando reduzir o erro de treino, eventualmente decorando ruído. Controlar M (ou usar early stopping) não é opcional em boosting da forma que é dispensável em Random Forest.

A figura a seguir mostra esse fenômeno de forma isolada: a perda de treino (azul) cai monotonicamente até quase zero, sem nunca "perceber" que está decorando ruído — mas a perda de validação (vermelho) atinge um mínimo por volta da rodada 24 e depois **piora continuamente** conforme o modelo continua ajustando os últimos resíduos, que a essa altura já são, em boa parte, ruído do conjunto de treino.



Sem early stopping, a perda de treino cai indefinidamente — mas a perda de validação atinge um mínimo e depois piora: o sintoma clássico de overfitting em boosting.

NO MERCADO

Um erro de mercado recorrente: um time treina um modelo de XGBoost com `n_estimators=1000` fixo (sem early stopping), obtém AUC alto no treino, e é surpreendido por desempenho ruim em produção. O diagnóstico quase sempre é este gráfico — a validação já tinha atingido seu melhor ponto centenas de rodadas antes, e o modelo em produção é a versão **overfitada**, não a versão ótima que existiu no meio do caminho.

4. XGBoost e LightGBM: os avanços de engenharia

Gradient Boosting clássico (o algoritmo acima) é lento e propenso a overfitting sem cuidados extras. XGBoost e LightGBM adicionam:

DEFINICAO

Regularização explícita na função objetivo: XGBoost adiciona uma penalidade sobre o número de folhas e a magnitude dos pesos das folhas — a mesma lógica de Ridge/Lasso (módulo 2), agora aplicada à estrutura da árvore, não a coeficientes lineares.

Boosting de segunda ordem (Newton boosting): em vez de usar só o gradiente (primeira derivada) para ajustar cada árvore, XGBoost usa também a Hessiana (segunda derivada, tema 2) — uma aproximação de Newton em vez de gradiente descendente simples, convergindo mais rápido e com passos mais bem calibrados por região.

Busca de corte por histograma: em vez de testar todo limiar possível (o algoritmo guloso exato do módulo de árvores), agrupa valores em compartimentos (*bins*) e testa só os limiares dos compartimentos — muito mais rápido, com perda de precisão desprezível na prática.

Crescimento por folha (LightGBM) vs. por nível (XGBoost padrão): LightGBM cresce a árvore expandindo sempre a folha com maior ganho potencial (*leaf-wise*), em vez de expandir todas as folhas de um nível antes de passar ao próximo (*level-wise*) — árvores mais eficientes para a mesma profundidade, mas com maior risco de overfitting em datasets pequenos, exigindo mais atenção a `min_child_samples`/profundidade máxima.

NO MERCADO

As duas bibliotecas também lidam **nativamente** com valores faltantes (aprendendo, durante o treino, para que lado do corte um valor ausente deveria ir) — uma vantagem prática real sobre a exigência de imputação explícita (tema 3) que outros modelos deste tema têm. Em um pipeline real de crédito ou fraude, onde é comum ter 10-20% de valores ausentes em algumas features (histórico de conta nova, por exemplo), essa capacidade evita uma etapa inteira de imputação que, mal feita, poderia introduzir viés (tema 3, módulo 2).

5. Early stopping: decidir M sem vaziar o teste

FORMULARIO

Divida o treino em treino-efetivo e validação. A cada rodada de boosting, meça a perda na validação. Pare quando a perda de validação não melhorar por k rodadas consecutivas (`early_stopping_rounds`). Isso escolhe M automaticamente, evitando tanto poucas árvores (underfitting) quanto árvores demais (overfitting) — e, corretamente implementado, nunca usa o conjunto de teste final nesse processo, só um conjunto de validação à parte (o mesmo princípio de vazamento de hiperparâmetro do módulo 2).

6. Erros que custam caro — checklist

- Treinar um número fixo e grande de árvores sem early stopping, arriscando overfitting silencioso (veja a figura da seção anterior).
- Usar taxa de aprendizado alta (`learning_rate` próximo de 1) esperando convergência rápida — o resultado costuma ser instável e pior.
- Usar o conjunto de teste final para decidir quando parar (early stopping) — use um conjunto de validação separado.
- Comparar Random Forest e boosting usando os hiperparâmetros padrão dos dois sem ajuste — boosting é mais sensível a hiperparâmetros mal escolhidos, e uma comparação "no padrão" costuma subestimar seu potencial.
- Usar árvores profundas como aprendizes fracos em boosting — o ponto de boosting é combinar muitos aprendizes **rasos**; árvores profundas tendem a overfitar rápido demais mesmo com poucas rodadas.
- Ignorar `min_child_samples`/profundidade em LightGBM (crescimento leaf-wise), que pode overfitar mais rápido que XGBoost em datasets pequenos sem essas restrições.

7. Para ir além

- Friedman (2001), *Greedy Function Approximation: A Gradient Boosting

Machine* — o artigo que formaliza Gradient Boosting como descida no espaço de funções.

- Chen & Guestrin (2016), *XGBoost: A Scalable Tree Boosting System* — o

artigo original de XGBoost, com a derivação completa da regularização e do boosting de segunda ordem.

- Ke et al. (2017), *LightGBM: A Highly Efficient Gradient Boosting

Decision Tree* — o artigo original de LightGBM, incluindo o crescimento leaf-wise e a busca por histograma.